

DOCUMENTATION TECHNIQUE

ObRail Europe

Mise en production d'une solution d'intelligence artificielle

MSPR TPRE532 · Bloc E6.3

Certification Professionnelle Développeur en Intelligence Artificielle et Data Science

RNCP36581

Équipe projet

Kouamé Johan BILÉ

Joseph HACCANDY

Glody KUTUMBAKANA

Nabil DIA

Date de soutenance : 20/06/2026

Table des matières

Table des matières	2
Chapitre 1 Introduction.....	2
1.1 Contexte du projet.....	2
1.2 Problématique	3
1.3 Objectifs opérationnels.....	4
1.4 Organisation du rapport	4
1.5 Structure du projet.....	4
Chapitre 2 Base de données	5
2.1 Modèle conceptuel	5
2.2 Choix de PostgreSQL	6
2.3 ETL Talend	7
Les propriétés ACID appliquées au projet ObRail	7
Les trois V du Big Data appliqués au projet ObRail.....	8
2.4 Flux de données.....	9
2.5 Conformité RGPD	10
Chapitre 3 Backend	11
3.1 Architecture technique	11
3.2 Choix techniques.....	12
3.3 Endpoints de l'API	13
3.4 Sécurité	14
3.5 Documentation automatique	15
Chapitre 4 Frontend	15
4.1 Architecture technique	15
4.2 Choix techniques.....	16
4.3 Les trois pages du tableau de bord	16
Page Trajets.....	16
Page Observatoire.....	17
Page Supervision.....	18
4.4 Accessibilité RGAA	19
4.5 Communication avec le backend	20
Chapitre 5 Tests et intégration continue.....	20
5.1 Types de tests	20
5.2 Organisation des tests	21
5.3 Outils de test	22
5.4 Pipeline GitHub Actions	22
5.5 Garde-fou local : pre-commit hooks.....	24
5.6 Couverture du code par les tests.....	25

5.7 De l'intégration continue à la livraison continue	26
Chapitre 6 Supervision.....	26
6.1 Pourquoi monitorer.....	26
6.2 Architecture Prometheus et Grafana	26
6.3 Métriques collectées	27
6.4 Le dashboard Grafana en six panneaux	27
6.5 SLI, SLO et SLA définir ce que signifie « ça marche »	29
6.6 Reproductibilité garantie	29
Chapitre 7 Déploiement	30
7.1 Déploiement local avec Docker Compose.....	30
7.2 Build et push des images	31
7.3 Options de déploiement en ligne	31
7.4 Mise en production effective sur Neon, Render et Streamlit Cloud	32
Chapitre 8 Conclusion.....	34
8.1 Synthèse des compétences couvertes	34
8.2 Perspectives d'accueil d'un futur modèle d'intelligence artificielle	35
Lien du repo github : github.com/JXPM/MSPR	36

Chapitre 1 Introduction

1.1 Contexte du projet

ObRail Europe est un observatoire indépendant fondé en 2018, dont la mission principale est d'analyser les flux ferroviaires à l'échelle du continent européen et de promouvoir la mobilité durable. L'organisation travaille en étroite collaboration avec les institutions européennes, notamment la Commission européenne et le Parlement européen, ainsi qu'avec des organisations non gouvernementales spécialisées dans la transition écologique des transports, comme Transport et Environnement et Back-on-Track. Elle entretient également des partenariats avec les grands opérateurs ferroviaires du continent, parmi lesquels la SNCF, ÖBB Nightjet ou encore Deutsche Bahn.

Le projet s'inscrit dans deux cadres stratégiques européens majeurs. D'une part, il répond aux objectifs du Pacte Vert européen, connu sous le nom de Green Deal, qui vise à réduire de 55 % les émissions de gaz à effet de serre d'ici 2030 et à décarboner le secteur des transports. D'autre part, il s'articule avec le programme TEN-T, le réseau transeuropéen de transport, qui cherche à renforcer les liaisons ferroviaires entre les pays membres de l'Union européenne.

Dans ce contexte, la question centrale posée par ObRail Europe est celle de la contribution respective des trains de jour et des trains de nuit à une mobilité interurbaine durable à l'échelle continentale. Les trains de nuit, en particulier, connaissent un regain d'intérêt depuis plusieurs années, car ils permettent de relier des métropoles éloignées sans recourir à l'avion, tout en limitant significativement les émissions de carbone. Comprendre leur répartition géographique, leur empreinte environnementale comparée à celle de l'avion et la manière dont les opérateurs les déploient est devenu un enjeu de politique publique.

1.2 Problématique

Le projet se heurte à plusieurs contraintes techniques. Les données ferroviaires européennes sont très dispersées : fichiers CSV issus d'OpenStreetMap, fichiers JSON publiés par Back-on-Track, flux GTFS fournis par la SNCF, fichiers Excel internes. Chaque opérateur publie dans son propre format, sans référentiel commun entre les pays.

La qualité de ces données est inégale : doublons, codes UIC manquants, fuseaux horaires incohérents, noms de villes mal encodés. À cela s'ajoute l'absence de standard transfrontalier pour harmoniser les informations d'un pays à l'autre.

Le traitement de ces données doit par ailleurs respecter le RGPD, qui impose une transparence totale sur les traitements effectués. Enfin, le projet est soumis à une contrainte temporelle forte : les résultats doivent être disponibles avant la fin de l'année pour alimenter les travaux du Parlement européen.

Le point de départ n'est pas non plus une page blanche. Un prototype avait été livré, reposant sur une base PostgreSQL et une API REST, mais son déploiement était entièrement manuel, aucun test automatisé n'était en place, et aucun mécanisme de supervision ne permettait de détecter les incidents. Il fonctionnait en environnement contrôlé, mais il n'était pas industrialisable.

La question est donc de savoir :

Comment passer d'un prototype fonctionnel à une application web industrialisée, testée, supervisée en temps réel, et capable d'accueillir un futur modèle d'intelligence artificielle ?

1.3 Objectifs opérationnels

Pour répondre à cette problématique, le groupe s'est fixé sept objectifs opérationnels .

Le premier : industrialiser la stack technique à l'aide de Docker et Docker Compose, afin de garantir une reproductibilité totale de l'environnement sur n'importe quelle machine.

Le deuxième : concevoir une interface professionnelle conforme aux exigences du Référentiel Général d'Amélioration de l'Accessibilité (RGAA), niveau AA.

Le troisième : mettre en place une stratégie de tests complète, couvrant les tests unitaires, d'intégration, de contrat, de qualité et de bout en bout.

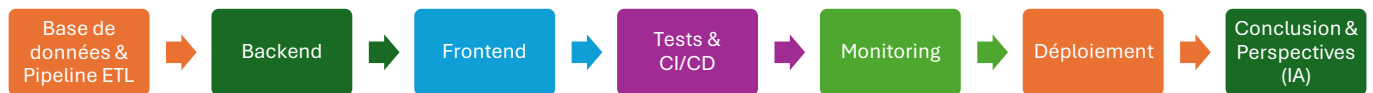
Le quatrième : automatiser l'intégration et la livraison continues via GitHub Actions.

Le cinquième : instrumenter le backend avec Prometheus et Grafana afin d'assurer une supervision des performances en temps réel.

Le sixième : garantir la mise en conformité réglementaire, notamment en matière de sécurité et de respect du RGPD.

1.4 Organisation du rapport

Ce rapport suit la structure logique du projet, en partant des fondations techniques pour remonter jusqu'aux couches supérieures.



1.5 Structure du projet

Le projet est organisé en plusieurs répertoires distincts, chacun correspondant à un composant fonctionnel. Cette structuration reflète la séparation des responsabilités mise en œuvre tout au long du développement.

MSPR3/	
├─ backend/	API REST FastAPI
│ └─ app/	
│ └─ main.py	Point d'entrée, middlewares, Prometheus
│ └─ database.py	Connexion SQLAlchemy
│ └─ models/	Classes ORM des 10 tables
│ └─ schemas/	Schémas Pydantic (validation)
│ └─ routes/	Endpoints FastAPI (5 fichiers)
│ └─ services/	Logique métier et requêtes SQL
│ └─ tests/	91 tests pytest (8 fichiers)
├─ dashboard/	Tableau de bord Streamlit
│ └─ app.py	Point d'entrée, navigation, styles RGAA
│ └─ _pages/	3 pages fonctionnelles
│ └─ components/	Graphiques Plotly, cartes Folium, icônes SVG
│ └─ services/	Client HTTP backend + requêtes Prometheus
│ └─ tests/	31 tests pytest
│ └─ tests_e2e/	6 tests Playwright
├─ monitoring/	Configuration Prometheus + Grafana
├─ talend/	Pipeline ETL (9 jobs compilés en JAR)
├─ .github/workflows/	Pipeline CI/CD (9 jobs GitHub Actions)
├─ docker-compose.yml	Orchestration des 5 services
└─ .env.example	Template variables d'environnement

Chapitre 2 Base de données

2.1 Modèle conceptuel

La base de données constitue le socle de toute l'application. Elle est structurée autour de dix tables organisées en trois couches logiques.

La première couche, dite couche référentiel, regroupe les données de structure du réseau ferroviaire. La table **pays** stocke les vingt-six pays européens, identifiés par leur code ISO à deux lettres. La table **gare** recense toutes les gares du réseau avec leur nom, leurs coordonnées GPS et leur appartenance à un pays. La table **opérateur** liste les compagnies ferroviaires. La table **ligne** décrit les lignes ferroviaires avec leur nom, leur distance et leur type, qui peut être *JOUR* ou *NUIT*. La table **type_train** référence les catégories de matériel roulant.

La deuxième couche, dite couche exploitation, couvre les circulations réelles. La table **trajet** représente un aller simple entre une gare de départ et une gare d'arrivée, avec les horaires de départ et d'arrivée stockés au format texte HH:mm:ss. La table **itineraire** détaille les arrêts intermédiaires de chaque trajet, ordonnés par leur rang de passage et rattachés à leurs coordonnées géographiques via le code UIC de chaque gare.

La troisième couche, appelée couche analyse, contient une unique table **emission**, qui stocke l'empreinte carbone de chaque trajet, à la fois pour le train et pour l'avion. Deux tables d'association complètent ce modèle : la table **exploite** lie les opérateurs aux lignes qu'ils exploitent, et la table **utilisation** associe les opérateurs aux types de matériel roulant qu'ils emploient.

Les clés primaires suivent des logiques différentes selon les tables : la table **pays** utilise le code ISO à deux caractères comme clé naturelle, la table **gare** utilise le code UIC à sept chiffres, la table **trajet** utilise un identifiant alphanumérique issu des données sources, et la table **itineraire** utilise une clé composite formée de l'identifiant du trajet et du rang de l'arrêt.

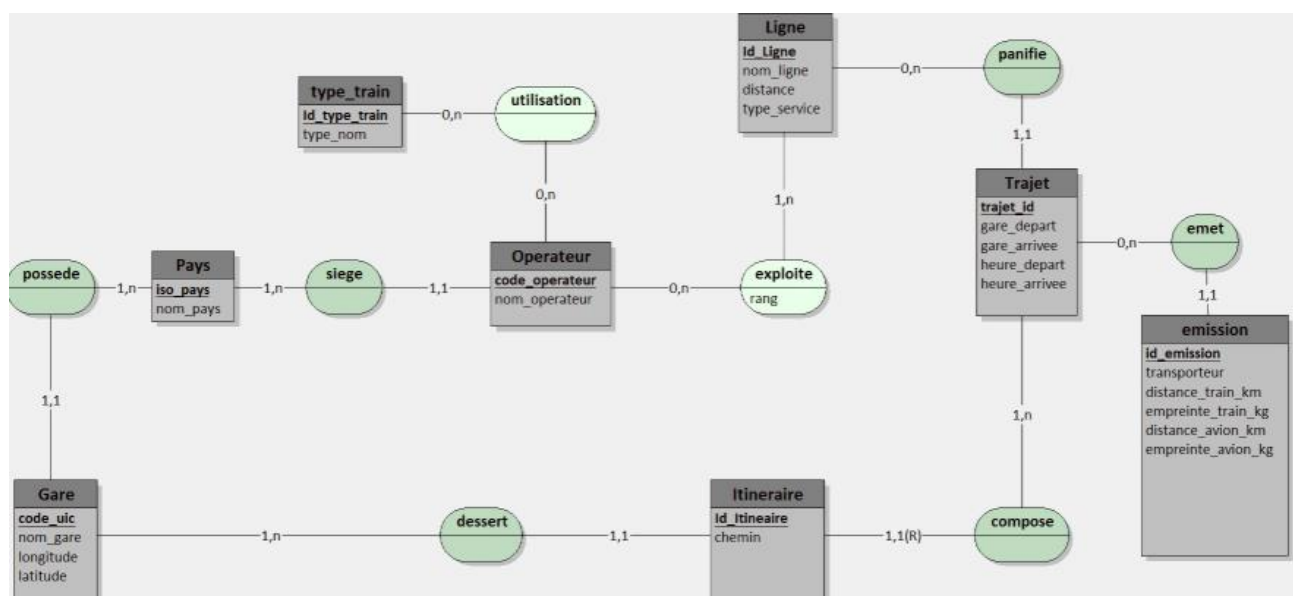


Figure 1 / Modèle de données

2.2 Choix de PostgreSQL

Le modèle respecte la troisième forme normale (3NF), qui est le standard de normalisation des bases de données relationnelles. La normalisation consiste à organiser les colonnes et les tables de façon à éliminer les redondances et à prévenir les anomalies de mise à jour. Concrètement, si le

nom d'une gare change, une seule mise à jour dans la table gare suffit pour que tous les trajets qui la référencent reflètent immédiatement le nouveau nom, sans avoir à parcourir l'ensemble des tables.

Le choix du système de gestion de base de données s'est porté sur PostgreSQL plutôt que sur ses concurrents pour plusieurs raisons complémentaires. Par rapport à MySQL, PostgreSQL offre un meilleur support des contraintes d'intégrité référentielle et une conformité plus stricte au standard SQL, ce qui garantit la cohérence des relations entre les onze tables. Par rapport à SQLite, PostgreSQL gère nativement la concurrence d'accès, ce qui est indispensable dès lors que plusieurs services Docker lisent simultanément la même base.

Par rapport à MongoDB, un système de base de données orientée documents, PostgreSQL impose un schéma relationnel strict qui correspond exactement à la nature des données ferroviaires, dans lesquelles chaque trajet appartient à une ligne, chaque ligne est exploitée par un opérateur, et chaque gare est localisée dans un pays. Enfin, PostgreSQL est le système de gestion de base de données de référence pour la bibliothèque SQLAlchemy que nous utilisons dans le backend, et il satisfait pleinement aux exigences de conformité ACID, acronyme désignant les propriétés d'Atomicité, Cohérence, Isolation et Durabilité des transactions.

2.3 ETL Talend

La Durabilité garantit que les données d'une transaction validée sont définitivement enregistrées, même en cas de panne. PostgreSQL y parvient grâce au mécanisme WAL (Write-Ahead Logging) : avant d'écrire dans les fichiers principaux, il consigne d'abord les opérations dans un journal. En cas de panne, ce journal permet de rejouer les opérations non encore écrites et de retrouver un état cohérent au redémarrage.

L'Isolation garantit que les transactions concurrentes ne s'interfèrent pas. Pendant qu'un job Talend insère de nouveaux trajets dans une transaction non encore validée, le backend FastAPI voit la base dans l'état précédant cette insertion et ne lit jamais de données partielles ou temporaires.

La Cohérence garantit que toute transaction amène la base d'un état valide à un autre état valide. Les contraintes de clés étrangères, d'unicité et de non-nullité déclarées dans le schéma sont vérifiées automatiquement à chaque écriture, sans qu'aucun code applicatif n'ait à s'en charger manuellement.

L'Atomicité garantit qu'une transaction est tout ou rien. Lorsque le job Talend insère un trajet, il insère également les enregistrements correspondants dans les tables itineraire et emission dans la même transaction. Si l'insertion de l'itinéraire échoue parce qu'un code UIC est absent, PostgreSQL annule automatiquement l'ensemble de la transaction, laissant la base dans un état cohérent. Sans cette propriété, on pourrait se retrouver avec un trajet sans itinéraire ni émissions associées.

Les propriétés ACID appliquées au projet ObRail

La Vitesse désigne la fréquence à laquelle les données arrivent et doivent être interrogées. En écriture, les neuf jobs Talend exécutent des insertions massives en lot lors de chaque chargement mensuel. En lecture, plusieurs services Docker interrogent simultanément la base : le backend FastAPI, les tests d'intégration du pipeline CI/CD, et les éventuels jobs ETL parallèles. PostgreSQL gère cette concurrence grâce à son mécanisme MVCC (contrôle de concurrence par multiversion) : chaque transaction de lecture voit un instantané cohérent de la base au moment où elle commence, sans bloquer ni être bloquée par les transactions d'écriture simultanées.

La Variété désigne l'hétérogénéité des formats et des structures de données. Notre projet est confronté à une variété maximale : fichiers CSV d'OpenStreetMap, fichiers JSON de Back-on-Track, flux GTFS de la SNCF au format ZIP, chacun avec ses propres conventions. Le schéma relationnel strict de PostgreSQL force une normalisation complète lors de l'insertion : une gare dont le code pays n'existe pas dans la table pays ne peut tout simplement pas être insérée, ce qui garantit la cohérence du référentiel.

Le Volume désigne la quantité totale de données à stocker et à interroger. Les données ferroviaires européennes représentent plusieurs dizaines de milliers de gares, de trajets et d'itinéraires couvrant vingt-six pays. PostgreSQL gère ce volume sans dégradation de performance grâce à son moteur d'indexation B-tree, qui permet des recherches en temps logarithmique même sur des tables contenant des millions d'enregistrements. SQLite, par comparaison, deviendrait sensiblement plus lent à mesure que le volume de données augmenterait avec les prochaines intégrations de sources européennes.

Les trois V du Big Data appliqués au projet ObRail

L'analyse s'est appuyée sur deux cadres complémentaires : les trois V du Big Data, qui caractérisent les contraintes imposées par les données elles-mêmes, et les propriétés ACID, qui garantissent la fiabilité des transactions.

L'ETL, acronyme d'Extract Transform Load désignant le pipeline de traitement des données, est orchestré avec Talend Open Studio for Data Integration. Talend lit les sources brutes, les nettoie, les transforme et les insère dans PostgreSQL via des connecteurs JDBC, le protocole standard de connexion Java aux bases de données.

Les sources de données mobilisées sont au nombre de trois principales. Le fichier `stations.csv` issu d'OpenStreetMap fournit les gares avec leurs coordonnées géographiques. Le fichier `trips.json` publié par Back-on-Track contient les trajets ferroviaires européens, notamment les trains de nuit, avec les informations d'opérateurs, de lignes, d'itinéraires et d'émissions de CO2. Les fichiers GTFS de la SNCF, au format ZIP contenant des fichiers CSV, fournissent les catégories de matériel roulant.

Le choix de Talend s'explique par sa capacité à traiter simultanément des formats multi-sources très hétérogènes, par sa traçabilité graphique qui permet de visualiser et de documenter chaque étape

de transformation, et par le fait que les jobs compilés sous forme de fichiers JAR sont réutilisables sans dépendance à l'environnement de développement.

Pour bien comprendre ce que fait concrètement chaque lettre de l'acronyme ETL, il est utile de détailler chaque phase. L'Extraction consiste à lire les données brutes depuis leurs sources sans les modifier : Talend ouvre le fichier `stations.csv`, parcourt le fichier `trips.json` et décompresse les fichiers GTFS dans leur état d'origine, avec tous leurs défauts — encodage hétérogène, valeurs manquantes, noms de colonnes différents d'une source à l'autre.

La Transformation est la phase la plus complexe. Pour chaque enregistrement extrait, Talend applique une série d'opérations : normalisation des encodages pour corriger les noms de villes mal encodés, déduplication pour supprimer les gares apparaissant plusieurs fois avec des orthographes légèrement différentes, conversion des types pour transformer les coordonnées GPS de chaînes de caractères en nombres flottants, et calcul des valeurs dérivées telles que l'empreinte carbone de l'avion.

Le Chargement est la phase finale : les enregistrements transformés et validés sont insérés dans PostgreSQL via JDBC en respectant l'ordre des dépendances entre tables — on ne peut pas insérer une gare avant que son pays soit présent, ni un trajet avant que ses gares de départ et d'arrivée existent. Cette contrainte d'ordre détermine la séquence des neuf jobs Talend.

Le pipeline Talend est composé de neuf jobs exécutés dans un ordre précis qui respecte les dépendances entre tables. Le premier job charge la table `pays` à partir des codes ISO. Le deuxième charge les gares depuis le fichier `stations.csv`. Le troisième charge les opérateurs ferroviaires. Le quatrième charge les types de matériel roulant depuis les données GTFS. Le cinquième charge les lignes. Le sixième charge les trajets. Le septième, le job `exploite`, établit les liens entre opérateurs et lignes. Le huitième charge les itinéraires, c'est-à-dire les arrêts intermédiaires. Le neuvième, enfin, calcule et insère les émissions de CO2.

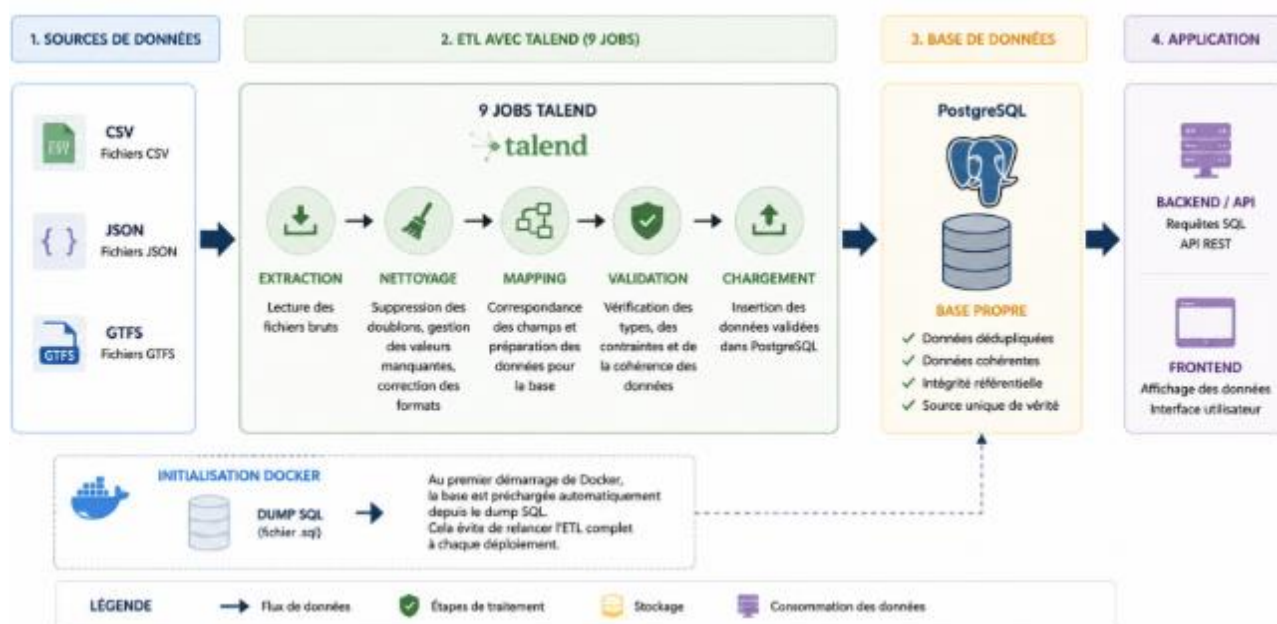


Figure 2 — Pipeline ETL Talend : 9 jobs dans l'ordre d'exécution

Le calcul de l'empreinte carbone de l'avion suit la formule : `empreinte_kg = distance_km × 0,158`. Ce facteur d'émission de 0,158 kilogramme de CO2 équivalent par passager et par kilomètre est issu des référentiels de l'ADEME, l'Agence de la transition écologique française, et du BEIS, le département britannique équivalent. L'empreinte carbone du train, quant à elle, est directement lue depuis le champ `emissions_co2e` du fichier `trips.json` fourni par Back-on-Track.

2.4 Flux de données

Le flux de données suit une chaîne linéaire et déterministe. Les sources brutes, qu'il s'agisse de CSV, de JSON ou de fichiers GTFS, sont lues par les neuf jobs Talend qui effectuent l'extraction, le nettoyage, le mapping et la validation de chaque enregistrement avant son insertion dans PostgreSQL. Ce processus aboutit à une base relationnelle propre, déduplicatée et cohérente, qui constitue la seule source de vérité pour le reste de l'application. Au premier démarrage de Docker, la base est préchargée automatiquement depuis le dump SQL versé dans le dépôt, ce qui évite de devoir relancer l'ETL complet à chaque déploiement.



2.5 Conformité RGPD

Le projet ne traite aucune donnée à caractère personnel. Toutes les données utilisées proviennent de l'open data public : les horaires de trains, les coordonnées des gares et les émissions de CO2 sont des informations publiques qui ne permettent pas d'identifier un individu. Plusieurs mesures techniques renforcent néanmoins la conformité réglementaire.

Les journaux applicatifs ne contiennent aucune adresse IP d'utilisateur. Les secrets de connexion à la base de données, comme les mots de passe, ne sont jamais versionnés dans le dépôt Git mais

sont gérés via un fichier d'environnement non versionné et via les GitHub Secrets pour le pipeline CI/CD. L'historique des métriques Prometheus est limité à trente jours par paramétrage explicite, ce qui évite l'accumulation indéfinie de données. Enfin, un fichier d'exemple documenté est fourni pour permettre à tout nouveau contributeur de configurer son environnement sans accéder aux vraies credentials.

Catégorie	Exemples concrets	Données personnelles ?
Données géographiques	Coordonnées GPS de gares, codes UIC	Non
Données opérationnelles	Horaires de trains, noms de lignes	Non
Données environnementales	Émissions CO2 par trajet en kg	Non
Données de référence	Codes pays ISO, noms d'opérateurs (entreprises)	Non
Données de traçabilité	URLs des sources, noms de fichiers	Non

Chapitre 3 Backend

3.1 Architecture technique

Avant de décrire les choix techniques, il est important de comprendre le rôle du backend dans l'architecture de l'application. Trois composants logiciels coexistent : la base de données PostgreSQL, qui stocke les informations de façon permanente ; le frontend Streamlit, qui est l'interface visible dans le navigateur ; et le backend, qui joue le rôle d'intermédiaire entre les deux.

Le frontend ne communique jamais directement avec la base de données. Cette conception évite d'exposer les identifiants de connexion sur le réseau et permet de centraliser les requêtes SQL complexes côté serveur. Le backend reçoit les demandes du frontend sous forme de requêtes HTTP, interroge PostgreSQL, formate les résultats en JSON et renvoie la réponse — le tout en quelques dizaines de millisecondes.

Une API REST (Representational State Transfer) organise ces échanges autour de ressources identifiées par des URL. Chaque URL correspond à une ressource du domaine ferroviaire : /trajets pour la liste des trajets, /gares pour les gares, /stats/emissions pour les statistiques d'émissions. La méthode HTTP GET indique que le client souhaite lire des données sans en modifier. L'ensemble des réponses est au format JSON, un format textuel structuré que tous les langages de programmation lisent nativement.

Le backend est une API REST, c'est-à-dire un serveur qui reçoit des requêtes HTTP et répond avec des données au format JSON. Son code est organisé en couches séparées dont chacune a une responsabilité précise.

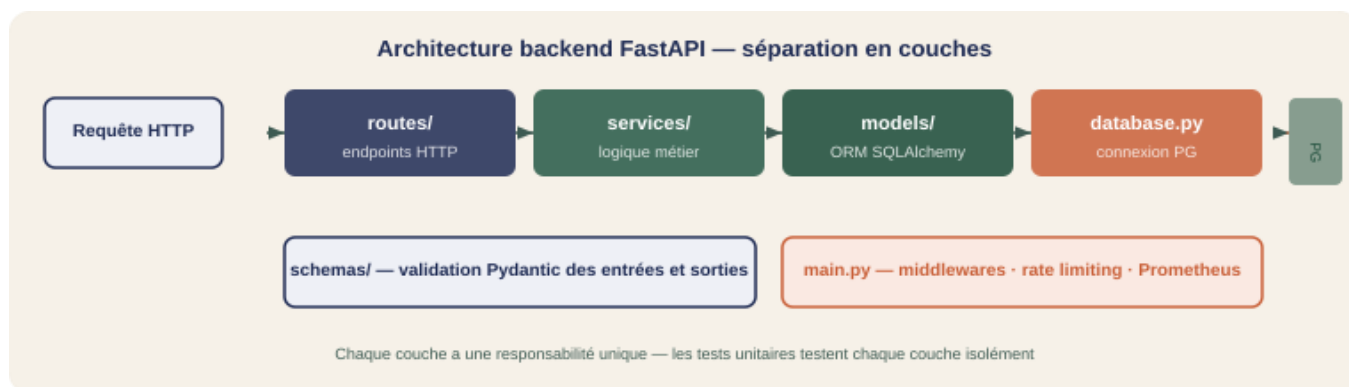


Figure 3 — Architecture interne du backend : séparation en couches

Le main.py : est le point d'entrée de l'application. C'est lui qui instancie le framework FastAPI, enregistre les middlewares de sécurité, active le rate limiting et connecte l'instrumentation Prometheus.

Le database.py : gère la connexion à PostgreSQL via SQLAlchemy et fournit une factory de sessions de base de données.

Le dossier models : regroupe les classes Python qui représentent les onze tables de la base de données. Chaque colonne SQL est modélisée comme un attribut Python typé.

Le dossier schemas : contient les schémas de validation Pydantic, qui définissent la forme exacte que doivent avoir les données entrantes et sortantes.

Le dossier routes : définit les endpoints de l'API, regroupés par domaine fonctionnel en cinq fichiers.

Le dossier services : contient la logique métier, notamment les requêtes SQL complexes et les fonctions de normalisation des noms de gares.

3.2 Choix techniques

Le Dans cette section, nous présentons les principaux choix technologiques retenus pour le développement de l'application.

Le choix de Python comme langage principal est naturel dans un projet de data science : son écosystème est le plus riche du domaine, avec des bibliothèques matures pour la manipulation de données, la validation et la création d'API.

Le choix de FastAPI comme framework d'API plutôt que Flask s'explique par deux avantages décisifs. D'une part, FastAPI génère automatiquement une documentation Swagger interactive à partir des annotations Python, ce qui évite de maintenir une documentation séparée pouvant diverger du code. D'autre part, FastAPI intègre nativement Pydantic pour la validation des données, ce qui permet de détecter immédiatement les erreurs de format avant même que la logique métier soit exécutée.

Le choix de SQLAlchemy : est utilisé comme ORM, c'est-à-dire comme traducteur automatique entre les objets Python et les requêtes SQL. C'est le standard de facto pour Python, et il prend en charge les relations complexes entre tables telles que celles de notre modèle.

SQLAlchemy est utilisé comme ORM (Object-Relational Mapper), c'est-à-dire comme traducteur automatique entre les objets Python et les requêtes SQL. Sans ORM, le code doit construire manuellement des chaînes de texte SQL, ce qui est fastidieux et vulnérable aux injections si les paramètres ne sont pas correctement échappés. Avec SQLAlchemy, les requêtes s'écrivent sous forme d'expressions Python lisibles, et les valeurs sont automatiquement paramétrées, éliminant ce vecteur d'attaque par construction.

Uvicorn sert de serveur ASGI (Asynchronous Server Gateway Interface). La différence essentielle avec un serveur WSGI traditionnel réside dans la gestion de la concurrence : un serveur WSGI crée un nouveau fil d'exécution pour chaque requête simultanée, ce qui devient coûteux en mémoire au-delà de quelques dizaines de requêtes. Un serveur ASGI comme Uvicorn peut gérer des milliers de connexions simultanées dans un seul processus grâce à la programmation asynchrone : une requête en attente d'une réponse de la base de données n'occupe pas un fil d'exécution entier pendant ce temps d'attente.

Le choix de Pydantic : assure le typage fort et la validation des données, garantissant que les données envoyées au frontend sont toujours dans le format attendu.

Le choix de Uvicorn : sert de serveur ASGI, un protocole asynchrone plus performant que le WSGI traditionnel, et il supporte les workers multiples pour gérer la concurrence.

3.3 Endpoints de l'API

L'API expose treize endpoints couvrant l'ensemble des besoins du dashboard et de la supervision.

Le endpoint GET /health : retourne simplement l'état de santé du serveur sous la forme d'un objet JSON confirmant que l'API répond correctement.

Le endpoint GET /metrics : expose les métriques de performance au format texte que Prometheus scrute toutes les quinze secondes afin de surveiller l'état du système.

Le endpoint GET /trajets : retourne la liste de tous les trajets disponibles dans la base de données.

Le endpoint GET /trajets/{id} : permet de récupérer le détail complet d'un trajet précis à partir de son identifiant.

Le endpoint GET /trajets/{id}/itineraire : renvoie les arrêts intermédiaires d'un trajet donné, dans l'ordre de passage.

Le endpoint GET /gares : fournit la liste de toutes les gares avec leurs coordonnées GPS, ce qui permet notamment leur affichage sur une carte.

Le endpoint GET /lignes : retourne la liste de toutes les lignes ferroviaires, en précisant leur type, qu'il s'agisse de lignes de jour ou de nuit.

Du côté des statistiques, le **endpoint GET /stats/trajets/count** : donne le nombre total de trajets disponibles dans le système.

Le endpoint GET /stats/trajets/type : affiche la répartition entre trains de jour et trains de nuit, permettant une vue globale de l'offre.

Le endpoint GET /stats/missions : compare l'empreinte carbone moyenne d'un trajet en train avec celle du transport aérien.

Le endpoint GET /stats/operateurs : présente le volume de trajets par opérateur ferroviaire, afin d'analyser leur répartition.

Le endpoint GET /stats/trajets/map : fournit les segments géographiques nécessaires pour tracer la carte complète du réseau.

Enfin, le **endpoint GET /docs** : donne accès à la documentation interactive Swagger générée automatiquement, facilitant la compréhension et les tests de l'API.

Les endpoints **/trajets** ainsi que l'ensemble des endpoints **/stats** sont soumis à un rate limiting, c'est-à-dire une limitation du nombre de requêtes sur une période donnée afin de protéger l'API contre les abus et les surcharges.

3.4 Sécurité

Deux mécanismes de sécurité sont implémentés directement dans main.py. Le premier est le rate limiting, mis en œuvre grâce à la bibliothèque `slowapi`. Il limite à soixante requêtes par minute et par adresse IP l'accès aux endpoints **/trajets** et **/stats**. Lorsque cette limite est dépassée, l'API répond

avec le code HTTP 429, qui signifie "Trop de requêtes", accompagné d'un message invitant à réessayer dans soixante secondes. Ce mécanisme protège l'API contre le scraping massif automatisé.

Le second mécanisme est l'injection automatique de quatre headers de sécurité HTTP dans chaque réponse de l'API. Le header `X-Content-Type-Options` avec la valeur `nosniff` empêche les navigateurs de deviner le type d'un fichier, ce qui bloque un vecteur d'attaque dit de MIME sniffing. Le header `X-Frame-Options` avec la valeur `DENY` interdit l'intégration de l'API dans une iframe, ce qui protège contre le clickjacking, une attaque consistant à superposer un contenu frauduleux sur l'interface légitime. Le header `X-XSS-Protection` active le filtre de protection contre les injections de scripts des anciens navigateurs. Le header `Referrer-Policy` contrôle quelles informations de provenance sont transmises lors des requêtes cross-origin.

3.5 Documentation automatique

FastAPI génère automatiquement une interface Swagger UI accessible à l'adresse `/docs`. Cette interface, toujours synchronisée avec le code source, permet de tester directement chaque endpoint depuis le navigateur, de visualiser les schémas de données attendus et retournés, et de comprendre le comportement de l'API sans lire le code source. C'est un avantage considérable pour l'intégration avec les partenaires d'ObRail Europe et pour les tests manuels pendant le développement.

default

GET	/metrics	Metrics	^
GET	/health	Health	^

Trajets

GET	/trajets/	Get Trajets	^
GET	/trajets/{trajet_id}	Get Trajet	^
GET	/trajets/{trajet_id}/itineraire	Get Trajet Itinéraire	^

Gares

GET	/gares/	Get Gares	^
-----	---------	-----------	---

Lignes

GET	/lignes/	Get Lignes	^
-----	----------	------------	---

Stats

GET	/stats/trajets/count	Count Trajets	^
GET	/stats/lignes/count	Count Lignes	^
GET	/stats/gares/count	Count Gares	^
GET	/stats/pays/count	Count Pays	^
GET	/stats/trajets/type	Trajets By Type	^
GET	/stats/emissions	Emissions Stats	^
GET	/stats/operateurs	Stats Operateurs	^
GET	/stats/trajets/map	Trajets Map	^

Chapitre 4 Frontend

4.1 Architecture technique

Le frontend est un tableau de bord analytique développé avec Streamlit, un framework Python qui génère automatiquement des interfaces web sans écrire de HTML, de CSS ni de JavaScript. Son organisation suit une séparation claire entre les responsabilités.

Le fichier `app.py` est le point d'entrée de l'application. Il initialise Streamlit, définit la barre de navigation entre les pages et injecte les styles globaux, notamment les variables CSS de couleur et les règles d'accessibilité. Le dossier `pages` contient les trois pages fonctionnelles de l'application. Le dossier `components` regroupe les composants réutilisables : `charts.py` génère les graphiques Plotly, `map.py` génère les cartes interactives Folium, et `icons.py` encapsule les icônes vectorielles

SVG. Enfin, le dossier `services` contient `api_service.py`, le client HTTP centralisé qui regroupe tous les appels vers le backend et vers Prometheus.

4.2 Choix techniques

Streamlit a été choisi pour sa capacité à produire rapidement des interfaces analytiques interactives sans nécessiter de compétences en développement web front-end. Son intégration native avec Pandas pour la manipulation des données, Plotly pour les graphiques et Folium pour les cartes géographiques en fait l'outil idéal pour un projet de data science. Il inclut son propre serveur web, ce qui simplifie le déploiement Docker.

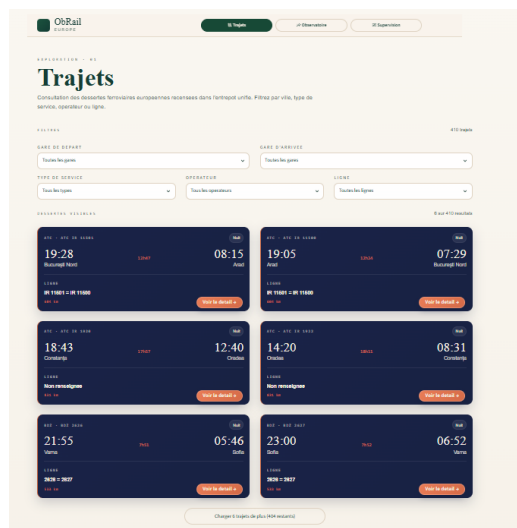
Ce choix présente cependant une contrepartie importante : Streamlit génère le HTML de façon automatique, ce qui limite le contrôle sur les attributs d'accessibilité comme les attributs aria. Cette limitation a été identifiée, documentée, et des mesures palliatives ont été mises en place, comme expliqué dans la section suivante.

Plotly a été retenu pour ses graphiques interactifs en Python, permettant aux utilisateurs de zoomer, de filtrer et d'inspecter les valeurs directement dans le navigateur. Folium permet de générer des cartes Leaflet interactives depuis Python, idéales pour visualiser les tracés des trajets ferroviaires sur une carte de l'Europe. Pandas assure la manipulation des données reçues de l'API avant leur affichage.

4.3 Les trois pages du tableau de bord

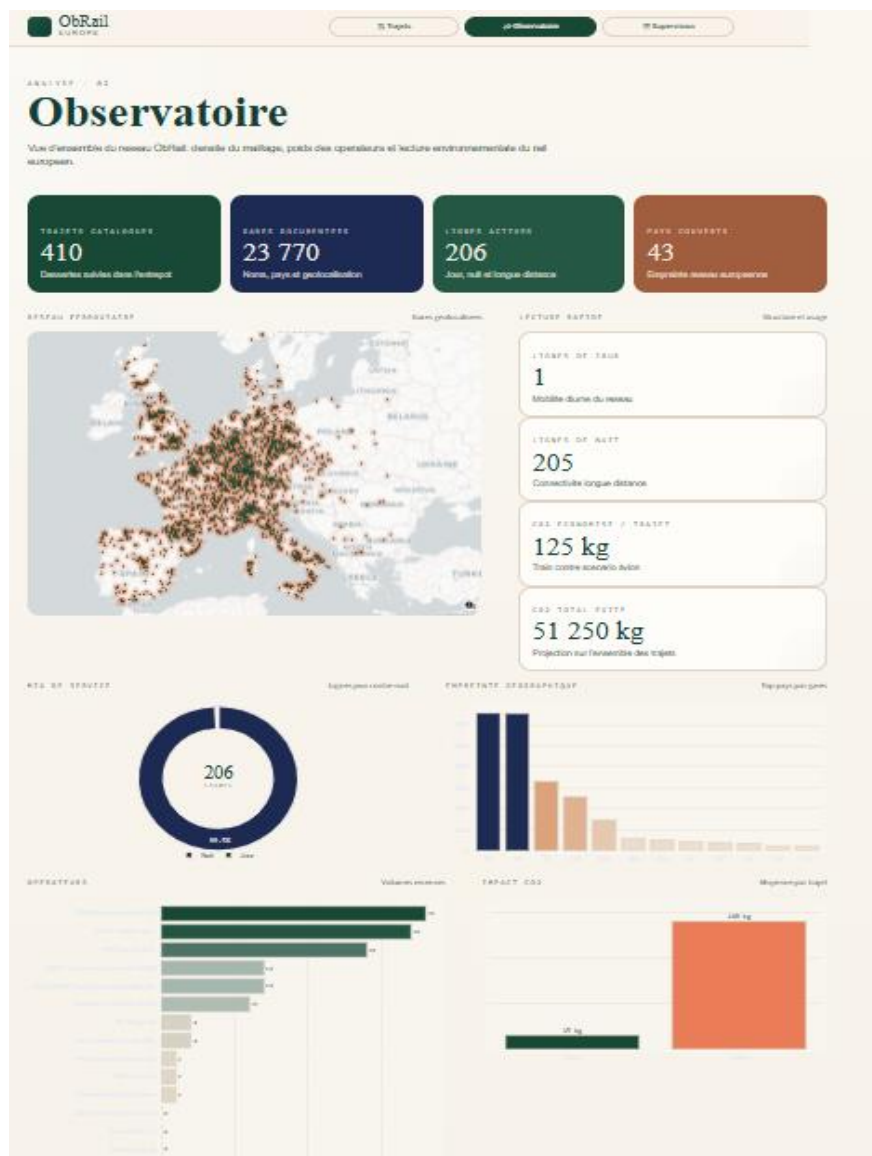
Page Trajets

La page Trajets est la page d'exploration détaillée du réseau. Elle propose des filtres permettant de sélectionner les trajets par pays, par opérateur et par type de service jour ou nuit. Elle affiche une carte interactive Folium qui trace le tracé géographique du trajet sélectionné, ses horaires de départ et d'arrivée, un graphique comparant les émissions de CO2 du train et de l'avion pour ce même trajet, ainsi que la liste ordonnée des arrêts intermédiaires de l'itinéraire.



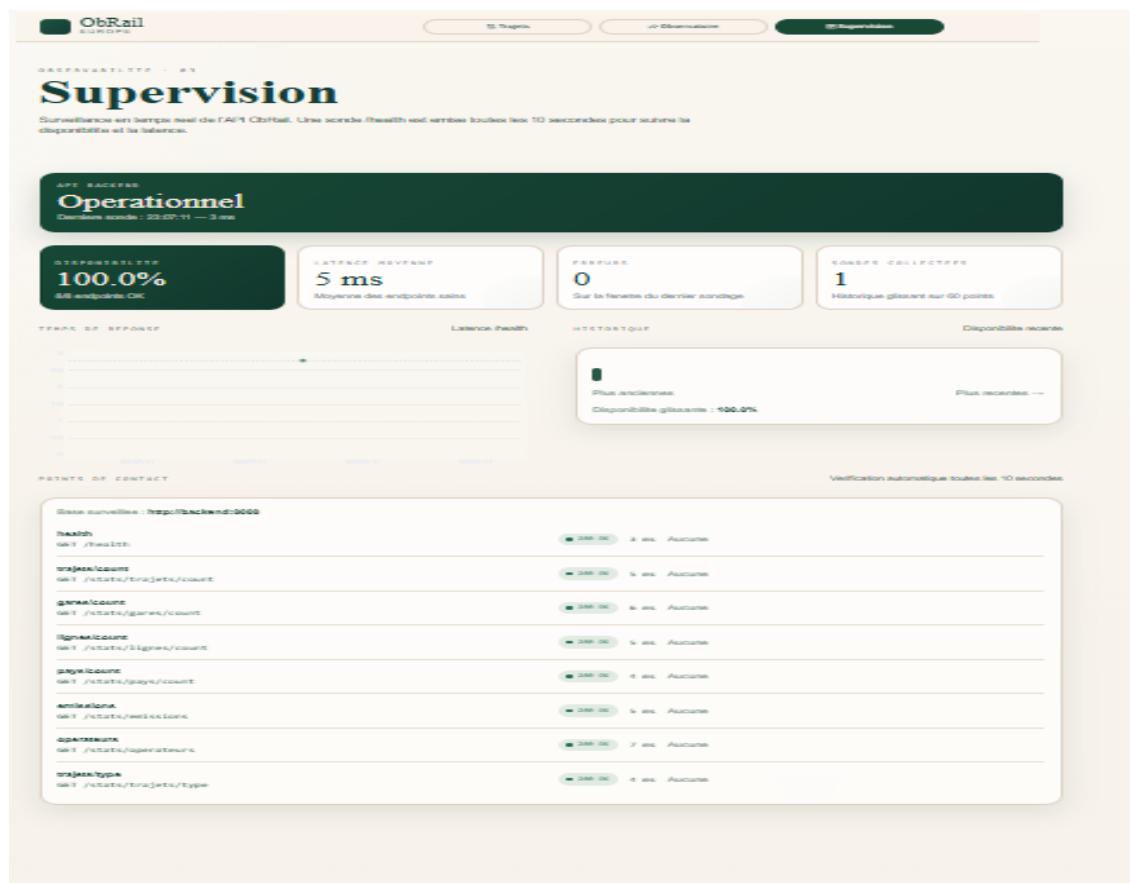
Page Observatoire

La page Observatoire est la vue d'ensemble synthétique destinée aux décideurs et aux partenaires institutionnels. Elle affiche en tête de page quatre indicateurs clés : le nombre total de trajets, le nombre de gares couvertes, le nombre de pays, et l'empreinte carbone moyenne comparée train versus avion. Elle présente ensuite un graphique en anneau montrant la répartition entre trains de jour et trains de nuit, un graphique à barres comparant les émissions de CO2 des deux modes de transport, et un histogramme représentant le volume de trajets par opérateur ferroviaire.



Page Supervision

La page Supervision est dédiée au suivi technique de l'état du système. Elle interroge l'endpoint `/health` du backend pour vérifier que l'API répond correctement et mesure sa latence. Elle interroge également Prometheus directement via des requêtes en langage PromQL pour récupérer le taux d'erreurs 5xx en temps réel et la latence au percentile 95. Si Prometheus est indisponible, la page affiche un état dégradé indiquant que les métriques ne sont pas accessibles, sans pour autant faire crasher l'application.



4.4 Accessibilité RGAA

L'application respecte plusieurs critères du Référentiel Général d'Amélioration de l'Accessibilité, niveau AA. Premièrement, le contraste entre le texte et le fond a été calculé et ajusté : la couleur dite muted, utilisée pour le texte secondaire, a été modifiée de la valeur `#4f6b62` à `#3d5a52`, ce qui

porte le ratio de contraste à 5,2 pour 1, au-dessus du seuil minimal de 4,5 pour 1 imposé par le niveau AA.

Deuxièmement, un lien d'évitement, dit skip link, est présent en tête de chaque page sous la forme d'un lien ancré vers l'identifiant `main-content`. Ce lien, invisible à l'écran mais accessible au clavier, permet aux utilisateurs naviguant au clavier ou avec un lecteur d'écran de sauter directement au contenu principal sans traverser toute la navigation. Troisièmement, la hiérarchie des titres est respectée : chaque page contient un seul titre de niveau h1, puis des titres h2 et h3 pour les sous-sections. Quatrièmement, la navigation au clavier est fonctionnelle avec un focus visible sur les éléments interactifs.

La principale limitation liée à Streamlit est l'impossibilité de contrôler les attributs aria dans le HTML généré automatiquement par le framework. Cette limitation a été documentée de façon transparente comme un compromis assumé, en accord avec l'esprit du RGAA qui reconnaît que certains environnements techniques imposent des contraintes.

4.5 Communication avec le backend

Toutes les communications avec le backend transitent par le module `api_service.py`, qui constitue un client HTTP centralisé et défensif. La fonction `get_trajets` appelle l'endpoint `/trajets` avec un timeout de dix secondes et retourne une liste vide en cas d'erreur réseau plutôt que de provoquer une exception. La fonction `get_stats_emissions` récupère les statistiques de CO2 et retourne un objet avec des valeurs à zéro en cas d'indisponibilité. La fonction `get_trajets_map` utilise un timeout de quinze secondes, plus long, car les données géographiques sont volumineuses. La fonction `ping` mesure la latence du backend en calculant le temps de réponse de l'endpoint `/health`, et retourne un objet contenant un booléen indiquant si le service répond et la latence en millisecondes.

La fonction `get_error_rate` interroge l'API de Prometheus via une requête PromQL calculant le taux de requêtes HTTP ayant retourné un code d'erreur 5xx sur les cinq dernières minutes. La fonction `get_api_latency_p95` interroge Prometheus pour calculer la latence au percentile 95 via la fonction `histogram_quantile` de PromQL. Ces deux fonctions retournent systématiquement un objet avec la clé `value` à `None` en cas d'erreur, sans jamais faire crasher le tableau de bord.

Chapitre 5 Tests et intégration continue

5.1 Types de tests

Pour comprendre la valeur d'une stratégie de tests, il est utile de se représenter ce qui se passe sans elle. La seule façon de vérifier qu'une modification du code n'a rien cassé serait de lancer l'application manuellement et de cliquer sur chaque fonctionnalité. Avec cent vingt-huit comportements à vérifier, cette approche prendrait des heures et resterait peu fiable — la mémoire humaine oublie inévitablement des cas limites.

Les tests automatisés résolvent ce problème : ce sont des programmes Python qui appellent les fonctions et les endpoints du projet, vérifient que les résultats correspondent aux valeurs attendues, et signalent immédiatement toute divergence. Une fois écrits, ces tests s'exécutent en quelques secondes à chaque modification du code, sans intervention humaine. Dans le contexte d'un projet industrialisé, les tests sont la condition sine qua non de la livraison continue : sans eux, il est impossible de garantir qu'une image Docker construite automatiquement est réellement déployable en production.

Le projet met en œuvre une stratégie de tests à cinq niveaux, chacun correspondant à une granularité différente de vérification.



Figure 4 — Stratégie de tests : 5 niveaux · 128 tests au total

Les **tests unitaires**, annotés avec le marqueur `pytest unit`, testent une seule fonction de façon totalement isolée, sans base de données ni serveur HTTP. Ils vérifient par exemple le comportement des fonctions de normalisation des noms de gares ou de génération des graphiques. Les **tests d'intégration**, annotés `integration`, testent un endpoint complet depuis la requête HTTP jusqu'à la réponse JSON, en utilisant une vraie base de données de test. Ils vérifient que les composants s'assemblent correctement.

Les **tests de contrat**, annotés `contract`, vérifient que la forme des réponses de l'API ne change jamais sans que les tests le signalent : si un champ disparaît ou change de type dans une réponse, le test échoue. Les **tests de qualité**, annotés `quality`, vérifient la cohérence métier des données issues de l'ETL Talend : absence de doublons, respect des formats attendus, cohérence géographique. Enfin, les **tests de bout en bout** utilisant Playwright pilotent un vrai navigateur web pour simuler des parcours utilisateur réels sur l'interface Streamlit.

5.2 Organisation des tests

La pyramide des tests, concept introduit par Mike Cohn, représente la distribution idéale des tests dans un projet : une large base de tests unitaires en bas, une couche intermédiaire de tests

d'intégration, et un sommet plus étroit de tests de bout en bout. Cette forme reflète un compromis entre vitesse d'exécution, coût de maintenance et niveau de confiance. Un test unitaire s'exécute en quelques millisecondes sans infrastructure. Un test d'intégration prend quelques secondes et nécessite une base de données de test. Un test de bout en bout prend plusieurs secondes à plusieurs minutes et nécessite la stack complète. Si on inversait cette pyramide en écrivant principalement des tests de bout en bout, la suite serait lente, fragile, et les développeurs finiraient par ne plus l'exécuter localement.

Dans notre projet, la pyramide est respectée : quatre-vingt-onze tests backend dont une majorité de tests unitaires et d'intégration, trente et un tests tableau de bord centrés sur les fonctions unitaires, et seulement six tests de bout en bout Playwright ciblant les parcours utilisateur les plus critiques.

Le projet compte au total cent vingt-huit tests répartis en trois suites. La suite backend comprend quatre-vingt-onze tests organisés dans huit fichiers. La suite dashboard comprend trente et un tests dans trois fichiers. La suite de tests de bout en bout comprend six tests dans deux fichiers.

Pour comprendre l'organisation des tests backend, il est essentiel de comprendre le rôle du fichier `conftest.py`. Dans `pytest`, ce fichier définit des fixtures — des ressources partagées entre plusieurs tests. Une fixture est une fonction décorée avec `@pytest.fixture` qui s'exécute automatiquement avant les tests qui en ont besoin et effectue le nettoyage après.

La fixture `db_session` crée avant chaque test une base de données SQLite en mémoire : une base temporaire qui vit uniquement dans la RAM du processus et disparaît à la fin du test. Elle est ensuite peuplée avec des données connues et maîtrisées — quatre pays, six gares, trois opérateurs, quatre trajets complets avec leurs itinéraires et émissions. Comme ces données sont réinitialisées à chaque test, chaque test part d'un état connu et propre, ce qui garantit l'isolation et la reproductibilité des résultats. L'ordre d'exécution des tests est donc sans importance, et aucun test ne peut polluer l'état d'un autre.

Le fichier `conftest.py` joue un rôle central dans la suite backend : il prépare avant chaque test une base de données SQLite en mémoire, une base temporaire légère qui mime le comportement de PostgreSQL pour les tests. Cette base est peuplée avec des données de test connues : quatre pays (France, Allemagne, Italie, Autriche), six gares (Paris Nord, Paris Lyon, Berlin Hbf, München Hbf, Milano Centrale, Wien Hbf), trois opérateurs (SNCF, ÖBB Nightjet, Deutsche Bahn), trois lignes (Paris-Berlin en train de jour, Paris-Vienne en train de nuit, Berlin-Milan en train de jour), quatre trajets avec leurs identifiants, leurs itinéraires et leurs émissions associées. Comme ces données sont réinitialisées à chaque test, chaque test part d'un état connu et propre, ce qui garantit l'isolation et la reproductibilité des résultats.

5.3 Outils de test

`pytest` a été retenu comme framework de test pour sa découverte automatique des fichiers de test, son système de marqueurs qui permet de filtrer les tests par type, et ses rapports détaillés incluant la couverture de code. Playwright a été choisi pour les tests de bout en bout car il pilote un vrai

navigateur Chromium en mode headless, c'est-à-dire sans interface graphique visible, et peut être exécuté dans l'environnement de CI/CD sans installation d'un serveur X11. Ruff sert de linter, c'est-à-dire d'outil d'analyse statique du code : il vérifie en quelques millisecondes que le code respecte les conventions de style Python et remplace avantageusement les outils plus anciens Flake8 et isort.

5.4 Pipeline GitHub Actions

Le pipeline d'intégration et de livraison continues est défini dans le fichier `.github/workflows/main.yml` et se déclenche automatiquement à chaque push sur les branches `main` et `develop` et à chaque Pull Request vers `main`. Il est composé de neuf jobs.

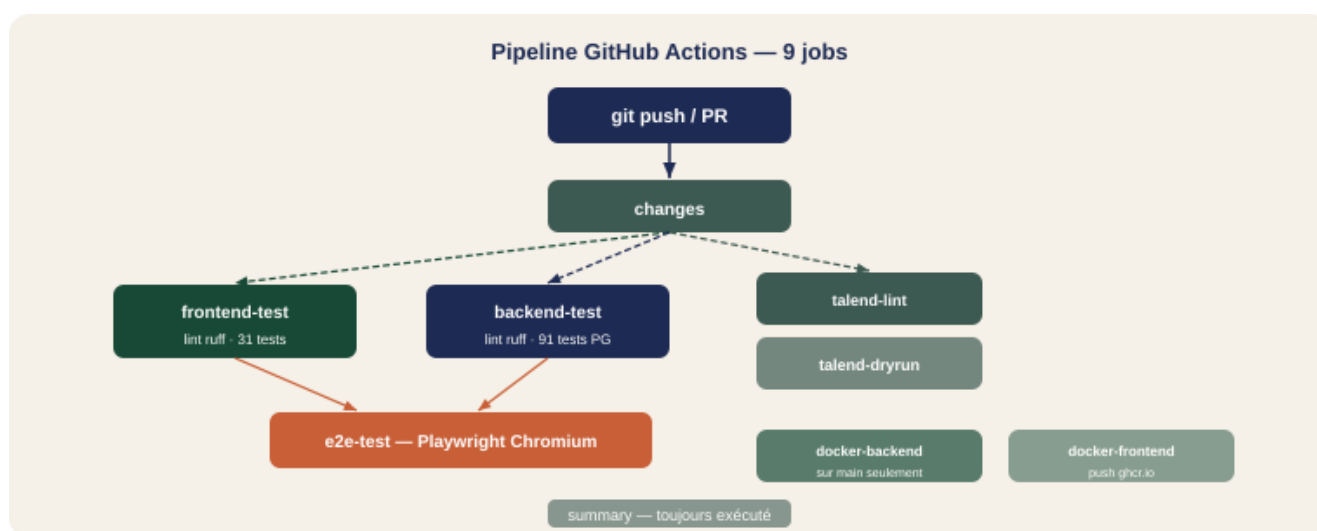


Figure 5 — Pipeline GitHub Actions : 9 jobs et leurs dépendances

Le job **changes** est le premier à s'exécuter. Il analyse quels dossiers ont été modifiés dans le commit, parmi `dashboard`, `backend` et `talend`, et transmet cette information aux jobs suivants. Ce mécanisme évite de retester l'intégralité de la suite quand un seul composant a changé.

Le job **frontend-test** s'exécute si des fichiers du dossier `dashboard` ont changé. Sur un serveur Linux Ubuntu temporaire fourni par GitHub, il installe Python 3.12, installe les dépendances du tableau de bord, puis lance Ruff pour vérifier la syntaxe du code avant de lancer les trente et un tests pytest du dashboard.

Le job **backend-test** s'exécute si des fichiers du dossier `backend` ont changé. Il démarre un service PostgreSQL de test, installe Python 3.12 et les dépendances du backend, lance Ruff pour le lint, puis exécute les quatre-vingt-onze tests pytest en utilisant cette base de données réelle.

Le job **talend-lint** valide l'intégrité du pipeline ETL. Il lance ShellCheck, un outil d'analyse des scripts shell, pour détecter les erreurs dans les scripts bash. Il effectue un scan de secrets pour vérifier qu'aucun mot de passe n'est présent en clair dans les scripts. Il vérifie la présence des neuf fichiers JAR correspondant aux neuf jobs Talend. Il vérifie enfin l'intégrité de chaque fichier JAR en le décompressant et en contrôlant que son contenu est lisible.

Le job **talend-etl-dryrun** exécute l'intégralité des neuf jobs Talend sur une base de test PostgreSQL provisionnée dans GitHub Actions, en repartant du dernier dump SQL disponible dans le dépôt. Les

logs de chaque job sont archivés comme artefact et conservés quatorze jours pour permettre le débogage.

Le job **e2e-test** s'exécute après la validation des tests frontend et backend. Il lance la stack Docker complète avec docker compose up, attend que Streamlit réponde sur le port 8501 via une boucle de vérification d'une durée maximale de soixante secondes, installe Playwright et son navigateur Chromium, puis exécute les six tests de bout en bout. Quelle que soit l'issue des tests, un docker compose down est systématiquement exécuté pour nettoyer l'environnement.

Les jobs **docker-frontend** et **docker-backend** s'exécutent exclusivement sur la branche main, après que les tests correspondants ont réussi. Ils construisent les images Docker du frontend et du backend depuis leurs Dockerfile respectifs, puis les poussent vers le GitHub Container Registry, le registre d'images Docker hébergé par GitHub. Chaque image est taguée avec trois références : le nom de la branche, le SHA du commit pour une traçabilité exacte, et le tag latest pour pointer vers la dernière version stable.

Le job **summary** s'exécute en dernier, quels que soient les résultats des autres jobs. Il génère un tableau récapitulatif affiché directement dans l'interface GitHub, montrant le statut de chaque bloc du pipeline.



5.5 Garde-fou local : pre-commit hooks

La pyramide des tests et le pipeline GitHub Actions garantissent qu'aucune régression ne passe la barrière du dépôt central. Mais ces deux dispositifs ne s'exécutent qu'à la création d'une Pull Request ou lors d'un push, c'est-à-dire après que le développeur a déjà rédigé son commit. Pour intercepter les défauts plus tôt, dès l'instant où le code est sur le point d'entrer dans l'historique Git, le projet met en place un troisième garde-fou : les pre-commit hooks.

Un pre-commit hook est un programme déclenché automatiquement par Git au moment d'un git commit. Si l'un des contrôles échoue, le commit est annulé et le développeur reçoit le détail de l'erreur dans son terminal. Cette boucle de retour locale permet de corriger les problèmes triviaux

— espaces en fin de ligne, fichier sans saut de ligne final, secret oublié dans un script — sans attendre les quelques minutes que prend la CI.

Le projet utilise le framework pre-commit, configuré via le fichier `.pre-commit-config.yaml` à la racine du dépôt. Sa philosophie est délibérée : chaque hook reflète exactement un contrôle effectué par la CI. Cette correspondance un-pour-un évite la situation, fréquente dans les projets mal configurés, où la CI échoue alors que le commit est passé en local — ou inversement.

Quatre familles de contrôles sont activées. Les contrôles d'hygiène, fournis par le dépôt officiel pre-commit-hooks, suppriment les espaces en fin de ligne, ajoutent un saut de ligne final aux fichiers qui en manquent, valident la syntaxe des fichiers YAML et JSON, détectent les marqueurs de conflit Git oubliés, refusent les fichiers de plus de deux mégaoctets et signalent les clés privées committées par mégarde. Les contrôles de style Python sont assurés par ruff, le même outil que la CI : il s'applique à `backend/app/` et à `dashboard/` à l'exclusion des dossiers de tests, avec correction automatique des erreurs réparables. Les scripts shell de l'ETL Talend sont validés par shellcheck, en niveau d'avertissement, exactement comme dans le job `talend-lint` de la CI. Enfin, un hook local en bash réplique la recherche de secrets en clair (motifs `password=`, `PGPASSWORD=`, `secret=`) dans les répertoires `talend/lancement`, `talend/planication` et `talend/exécution_etl`.

Une attention particulière est portée à la liste d'exclusions globale, déclarée dans le bloc `exclude` en tête de fichier. Sont ignorés les fichiers générés par Talend (`talend/Jobs/`), les dumps SQL massifs (`talend/dump/`), les scripts batch Windows (`*.bat` et `*.ps1`) ainsi que les documents bureautiques (`documentations/`). Cette précaution est essentielle : sans elle, les hooks d'hygiène réécriraient les fins de ligne CRLF des fichiers Windows en LF Unix, ce qui casserait leur exécution sur les machines de l'équipe travaillant sous Windows.

L'activation est triviale pour un nouveau contributeur : `pip install pre-commit && pre-commit install`. La première commande installe l'outil, la seconde inscrit le hook dans le répertoire `.git/hooks` du dépôt local. À partir de cet instant, chaque commit déclenche automatiquement la suite de contrôles. Le développeur peut également exécuter manuellement l'ensemble des hooks sur tous les fichiers du dépôt avec `pre-commit run --all-files`, ce qui est utile lors de l'initialisation ou après une mise à jour majeure de la configuration.

Cette mise en place complète la stratégie qualité du projet en trois cercles concentriques : les pre-commit hooks filtrent au niveau du commit individuel, les tests pytest et Playwright filtrent au niveau de la branche locale, et le pipeline GitHub Actions filtre au niveau de l'intégration centrale. Aucun défaut n'a vocation à franchir simultanément ces trois barrières.

5.6 Couverture du code par les tests

Le nombre de tests, à lui seul, ne suffit pas à caractériser la qualité d'une suite. Une centaine de tests qui n'exercent que le chemin nominal d'une seule fonction laissent l'essentiel du code sans filet de sécurité. La métrique pertinente est la couverture de code, c'est-à-dire le pourcentage de lignes du projet effectivement exécutées au moins une fois lorsque la suite de tests est lancée. Le projet mesure cette couverture grâce à `pytest-cov`, une extension de `pytest` qui s'appuie sur la bibliothèque `coverage.py`. À chaque exécution de la suite, l'outil instrumente le code source, enregistre les lignes traversées, et produit un rapport synthétique. Ce rapport est généré localement lors des exécutions manuelles et archivé comme artefact dans le pipeline GitHub Actions, conservé quatorze jours pour permettre le suivi de l'évolution dans le temps.

Sur la suite `backend`, qui regroupe quatre-vingt-onze tests, la couverture atteint 98 % sur un total de trois cent quatre-vingt-huit instructions du dossier `app/`. Seules neuf instructions ne sont pas exercées, réparties principalement dans le module `app/database.py`, qui contient le code de gestion d'une connexion à PostgreSQL en environnement de production que les tests, eux, remplacent par une base SQLite en mémoire. Tous les modèles SQLAlchemy, tous les endpoints des routes, et la quasi-totalité des services métier sont couverts à 100 %.

Sur la suite `dashboard`, qui regroupe trente et un tests, la couverture atteint 67 %. Cette valeur, plus modeste, s'explique par la nature même du tableau de bord : les composants de génération de cartes Folium dans `components/map.py` et certaines fonctions du client HTTP dans `services/api_service.py` sont exercés indirectement par les six tests Playwright de la suite `e2e`, qui pilotent un vrai navigateur et chargent réellement les pages. Les outils de mesure de couverture ne

traversent pas cette frontière entre Python et le navigateur, si bien que ce code apparaît à tort comme non couvert dans le rapport unitaire alors qu'il est validé fonctionnellement.

Les tableaux suivants détaillent la répartition fichier par fichier.

Couverture backend (98 % au total)

Module	Instructions	Manquantes	Couverture
app/main.py	51	1	98 %
app/database.py	14	4	71 %
app/models/ (8 fichiers)	81	0	100 %
app/routes/ (5 fichiers)	108	2	98 %
app/services/ (4 fichiers)	95	2	98 %
app/schemas/ (3 fichiers)	30	0	100 %
Total	388	9	98 %

Couverture dashboard (67 % au total)

Module	Instructions	Manquantes	Couverture
components/charts.py	60	0	100 %
components/icons.py	5	0	100 %
components/map.py *	26	26	0 %
services/api_service.py *	87	33	62 %
Total	178	59	67 %

* exercé fonctionnellement par les six tests Playwright e2e qui n'instrumentent pas le code unitaire. Une couverture de 98 % sur le backend constitue un seuil largement supérieur aux pratiques courantes de l'industrie, qui considère 80 % comme un objectif solide pour un projet en production. Elle reflète la stratégie d'écriture de tests adoptée dès le début du projet : pour chaque endpoint nouvellement ajouté, au moins un test d'intégration valide le chemin nominal, un test de contrat valide la forme de la réponse, et un ou plusieurs tests unitaires valident les fonctions de service appelées en interne. Cette discipline, soutenue par la barrière du pipeline CI qui rejette toute Pull Request faisant chuter la couverture, garantit que la qualité ne se dégrade pas avec le temps.

5.7 De l'intégration continue à la livraison continue

L'intégration continue et la livraison continue sont dans le même pipeline, mais s'appliquent à des contextes différents. Sur une Pull Request, seuls les tests sont exécutés : le code soumis doit passer l'intégralité des cent vingt-huit tests avant de pouvoir être fusionné dans la branche principale. Sur un push direct sur main, si les tests réussissent, les images Docker sont automatiquement construites et poussées vers le registre. Cette architecture garantit que l'image déployée en production est exactement celle qui a été testée, sans aucune intervention manuelle entre les deux étapes.

Chapitre 6 Supervision

6.1 Pourquoi monitorer

La supervision d'une application en production est aussi importante que le code lui-même. Sans monitoring, une panne du backend peut rester inaperçue pendant des heures, jusqu'à ce qu'un utilisateur signale le problème. Avec Prometheus et Grafana, les équipes d'ObRail Europe peuvent consulter en temps réel l'état de l'API, détecter les dégradations de performance avant qu'elles affectent les utilisateurs, et réagir rapidement aux incidents.

6.2 Architecture Prometheus et Grafana

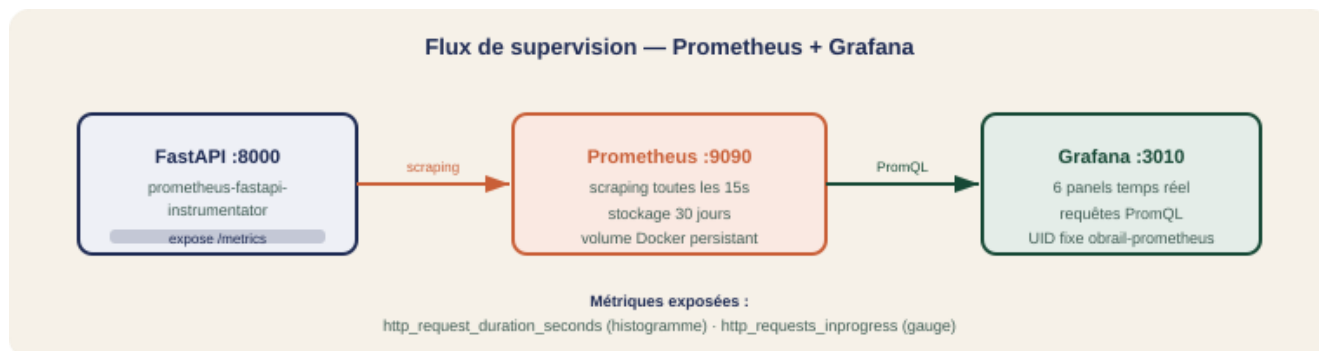


Figure 6 — Flux de supervision : FastAPI → Prometheus → Grafana

L'architecture de supervision repose sur deux outils complémentaires. Prometheus est un système de collecte de métriques open source qui fonctionne en mode scraping : il interroge périodiquement les services qu'il surveille pour récupérer leurs métriques. Dans notre projet, Prometheus scrute l'endpoint `/metrics` du backend toutes les quinze secondes. Grafana est l'outil de visualisation qui interroge Prometheus via le langage PromQL pour afficher les métriques sous forme de graphiques interactifs.

La liaison entre le backend FastAPI et Prometheus est assurée par la bibliothèque `prometheus-fastapi-instrumentator`. Cette bibliothèque injecte automatiquement un middleware dans FastAPI, c'est-à-dire une couche intermédiaire invisible qui mesure chaque requête reçue par l'API : sa durée, son code de réponse HTTP et l'endpoint appelé. Elle crée également l'endpoint `/metrics` que Prometheus interroge. Elle est activée en deux lignes de code dans `main.py`. Prometheus stocke l'historique de ces métriques dans un volume Docker persistant avec une rétention configurée à trente jours.

6.3 Métriques collectées

FastAPI expose nativement deux métriques grâce à l'instrumentation. La première est `http_request_duration_seconds`, un histogramme qui enregistre la durée de chaque requête en secondes, ventilée par endpoint et par code de réponse HTTP. La seconde est `http_requests_inprogress`, un compteur en temps réel du nombre de requêtes en cours de traitement à l'instant T.

Prometheus les agrège ensuite pour calculer des métriques dérivées plus utiles pour l'analyse opérationnelle : le taux de requêtes par seconde, le taux d'erreurs 4xx correspondant aux erreurs côté client et le taux d'erreurs 5xx correspondant aux erreurs côté serveur, les percentiles de latence aux niveaux p50, p95 et p99, et l'indicateur de disponibilité du service qui indique si le backend répond ou non.

6.4 Le dashboard Grafana en six panneaux

Le dashboard Grafana intitulé ObRail FastAPI Metrics présente six panneaux complémentaires, chacun répondant à une question opérationnelle précise.

Le premier panneau, **Requêtes par seconde**, utilise la requête PromQL `rate` appliquée au compteur de requêtes sur la dernière minute. Il permet de visualiser le trafic entrant en temps réel et de détecter des pics ou des baisses anormales d'activité.

Le deuxième panneau, **Taux d'erreurs 4xx et 5xx**, distingue visuellement les erreurs client, affichées en orange, des erreurs serveur, affichées en rouge. Cette distinction est essentielle car les erreurs 4xx indiquent généralement un problème dans l'utilisation de l'API par les clients, tandis que les erreurs 5xx signalent des bugs ou des indisponibilités côté serveur.

Le troisième panneau, **Latence p50/p95/p99**, utilise la fonction `histogram_quantile` de PromQL pour afficher les percentiles de latence. L'utilisation des percentiles plutôt que de la moyenne est un choix délibéré et important : la moyenne peut masquer des situations problématiques. Si quatre-vingt-dix-neuf requêtes répondent en dix millisecondes mais qu'une requête met dix secondes, la moyenne semble acceptable alors que le percentile 99 révèle immédiatement qu'un pourcent des utilisateurs attend dix secondes.

Le quatrième panneau, **Requêtes en cours**, affiche en temps réel le compteur `http_requests_inprogress`. Un pic inhabituel de requêtes simultanées peut signaler une saturation du backend ou une boucle de requêtes non maîtrisée.

Le cinquième panneau, **Total requêtes**, affiche le compteur cumulé de toutes les requêtes traitées depuis le démarrage du service. C'est un indicateur de volume d'utilisation global. Le sixième panneau, **Backend UP/DOWN**, est le plus immédiat : il affiche une pastille verte si le service répond et une pastille rouge sinon, en utilisant la métrique `up` de Prometheus qui vaut 1 quand le scraping réussit et 0 sinon.



6.5 SLI, SLO et SLA définir ce que signifie « ça marche »

La supervision technique suppose d'abord de définir précisément ce que signifie le fait qu'un service fonctionne correctement. Le domaine de la fiabilité des sites (Site Reliability Engineering, SRE) propose trois concepts complémentaires à cette fin.

Un SLI (Service Level Indicator) est une mesure quantitative d'un aspect du comportement du service. Dans notre projet, trois SLI sont instrumentés directement dans Prometheus : la disponibilité (métrique up qui vaut 1 si le backend répond au scrutage et 0 sinon), la latence (histogramme http_request_duration_seconds) et le taux d'erreurs (proportion de requêtes retournant un code HTTP 5xx).

Un SLO (Service Level Objective) est la valeur cible fixée en interne pour un SLI. Pour ObRail Europe, on pourrait définir : une disponibilité de 99 % sur trente jours glissants (moins de sept heures d'indisponibilité par mois), une latence au percentile 95 inférieure à 500 millisecondes pour les endpoints de statistiques, et un taux d'erreurs 5xx inférieur à 1 % sur une fenêtre de cinq minutes. Ces seuils sont directement observables dans le tableau de bord Grafana.

Un SLA (Service Level Agreement) est l'engagement contractuel formel vis-à-vis d'un partenaire externe, avec des conséquences définies en cas de non-respect. Dans le contexte d'ObRail Europe, il serait négocié avec la Commission européenne ou les opérateurs ferroviaires partenaires. Le SLA est généralement légèrement moins ambitieux que le SLO interne, pour laisser une marge de manœuvre entre la réalité opérationnelle et l'engagement public. Cette distinction SLI/SLO/SLA est importante car elle distingue ce qu'on mesure, ce qu'on vise, et ce qu'on promet — trois questions qui impliquent des acteurs différents.

6.6 Reproductibilité garantie

Lors de la mise en place du monitoring, un problème de reproductibilité a été identifié et documenté dans le fichier `why.md`. Grafana génère un identifiant unique aléatoire, appelé UID, pour chaque source de données au premier démarrage. Sur une deuxième machine, cet identifiant est différent. Or le fichier JSON du dashboard référence cet UID pour savoir quelle source de données utiliser. Résultat : les panneaux du dashboard s'affichaient vides sur toute machine autre que celle où le dashboard avait été initialement créé.

La solution adoptée consiste à fixer explicitement l'UID de la source de données Prometheus à la valeur littérale `obrail-prometheus` dans le fichier de provisioning Grafana. Le dashboard JSON référence ensuite cet UID fixe et non plus un identifiant aléatoire. Grâce au provisioning automatique via les volumes Docker, qui charge les dashboards et les datasources au démarrage depuis des fichiers du dépôt, le dashboard s'affiche correctement sur n'importe quelle machine, en CI comme en déploiement, sans la moindre intervention manuelle.

Chapitre 7 Déploiement

7.1 Déploiement local avec Docker Compose

Avant de décrire le déploiement concret, il est utile de situer notre architecture dans le cadre de la méthodologie 12-factor, formulée par Adam Wiggins de Heroku en 2011. Cette méthodologie définit douze principes qui, lorsqu'ils sont respectés, garantissent qu'une application web est déployable de façon fiable dans n'importe quel environnement.

Plusieurs de ces facteurs sont directement respectés dans notre projet. La base de code unique signifie que l'ensemble du projet — backend, frontend, configuration Prometheus, Dockerfiles et pipeline CI/CD — est versionné dans un seul dépôt Git. Les dépendances explicites sont garanties par les fichiers `requirements.txt` et les conteneurs Docker, sans supposer qu'une bibliothèque soit préinstallée sur le système hôte. La configuration dans l'environnement est assurée par les variables d'environnement injectées via `.env` en local et via les GitHub Secrets en CI/CD.

Le principe des processus sans état est respecté par le backend FastAPI : chaque requête est traitée indépendamment, sans que le serveur conserve de session entre les appels. Le principe de liaison de port est respecté : chaque service expose ses fonctionnalités via un port réseau déclaré dans `docker-compose.yml`. La jetabilité est garantie par les conteneurs Docker qui démarrent en quelques secondes et s'arrêtent proprement à réception du signal `SIGTERM`. Cette conformité à la méthodologie 12-factor garantit que l'application peut être déployée sur Fly.io, Render ou Oracle Cloud sans modification du code, uniquement en adaptant les variables d'environnement.

Le déploiement local de la stack complète s'effectue avec une seule commande : `docker compose up -d --build`. Docker lit le fichier `docker-compose.yml` qui définit les cinq services du projet et les assemble dans un réseau privé nommé `obrail`.

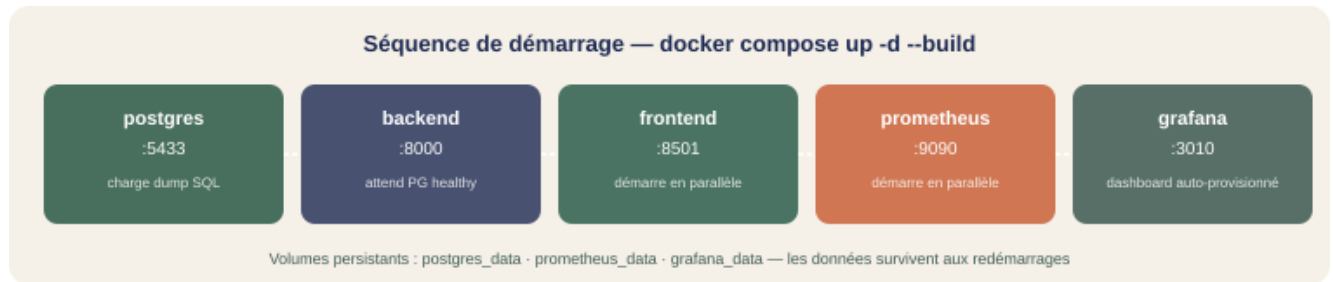


Figure 7 — Séquence de démarrage des 5 services Docker

Le service PostgreSQL démarre en premier et charge automatiquement le dump SQL du dépôt, ce qui initialise toutes les tables avec les données réelles sans intervention manuelle. Le service backend attend que PostgreSQL soit pleinement opérationnel grâce au mécanisme de condition de démarrage `depends_on` avec la vérification de santé `service_healthy`. Une fois PostgreSQL prêt, le backend démarre et expose l'API sur le port 8000. Le frontend Streamlit, Prometheus et Grafana démarrent ensuite de façon parallèle.

Des volumes Docker persistants garantissent que les données survivent aux redémarrages : `postgres_data` conserve les données de la base, `prometheus_data` conserve l'historique des métriques sur trente jours, et `grafana_data` conserve les configurations. Les ports exposés sur la machine hôte sont le port 5433 pour PostgreSQL, le port 8000 pour le backend, le port 8501 pour le frontend, le port 9090 pour Prometheus et le port 3010 pour Grafana. Le port non standard 5433 pour PostgreSQL est un choix délibéré pour éviter les conflits avec une éventuelle instance PostgreSQL locale déjà en cours.

Le premier démarrage nécessite trois à cinq minutes en raison du téléchargement des images Docker et de la construction des images custom. Les démarrages suivants prennent trente secondes environ car les images sont mises en cache localement.

Containers

[View feedback](#)

Container CPU usage ⓘ

1.27% / 1600% (16 CPUs available)

Container memory usage ⓘ

609.12MB / 7.4GB

Show charts

🔍

Search

☰

☑

Only show running containers

Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Memory (%)	Disk read/write	Net	Actions
● mspr3	-	-	-	1.24%	609.12MB / 37.9%	7.85%	543.8MB / 45.04%	49.3%	🔍 ⋮ 🗑
● obrail-postgres	307154c6e8c7	postgres:15	5433:5432 🔗	0%	42.31MB / 7.58Gi	0.55%	51.6MB / 623KB	1.3%	🔍 ⋮ 🗑
● obrail-prometheus	c68f06cfe16e	prom/prom	9090:9090 🔗	0.08%	48.31MB / 7.58Gi	0.62%	91.6MB / 42MB	27.1%	🔍 ⋮ 🗑
● obrail-grafana	0e65c9b596c8	grafana/gr	3010:3000 🔗	0.06%	63.3MB / 7.58GB	0.82%	163MB / 2.43MB	74.1%	🔍 ⋮ 🗑
● obrail-backend	26bcc924c450	mspr3-back	8000:8000 🔗	0.74%	311.3MB / 7.58Gi	4.01%	88.6MB / 0B	9.3%	🔍 ⋮ 🗑

Showing 6 items

7.2 Build et push des images

Sur un push sur la branche `main`, après que les tests ont réussi, le pipeline GitHub Actions exécute automatiquement les jobs `docker-backend` et `docker-frontend`. Ces jobs construisent les images

Docker depuis les Dockerfile de chaque composant, les taggent avec trois formats différents — le nom de la branche main pour les déploiements stables, le SHA du commit pour la traçabilité exacte de la version déployée, et le tag `latest` pour pointer vers la dernière version stable. Les images sont poussées vers GitHub Container Registry, accessible à l'adresse `ghcr.io`. Cette approche permet de déployer la dernière version validée en production avec une simple commande `docker pull`, sans avoir à reconstruire l'image localement.

7.3 Options de déploiement en ligne

Trois options de déploiement en ligne ont été étudiées pour héberger l'application publiquement. Fly.io est l'option recommandée : son offre gratuite inclut trois machines virtuelles partagées et une base PostgreSQL de trois gigaoctets, sans mise en veille automatique. Le déploiement s'effectue avec quatre commandes : la création de la base PostgreSQL, l'initialisation de l'application, l'attachement de la base à l'application, la configuration des secrets d'environnement, et le déploiement proprement dit.

Render est plus simple à prendre en main grâce à son interface web, mais présente un inconvénient majeur pour une démo publique : les services gratuits se mettent en veille après quinze minutes d'inactivité, ce qui rend le premier appel suivant cette mise en veille très lent, de l'ordre de trente secondes.

Oracle Cloud Free Tier offre deux machines virtuelles gratuites à vie, ce qui est techniquement suffisant pour faire tourner la stack complète incluant Prometheus et Grafana. En revanche, cette option nécessite une configuration plus technique, notamment l'ouverture manuelle des ports réseau dans les règles de sécurité du cloud et l'installation de Docker via SSH. Fly.io représente le meilleur compromis entre simplicité de déploiement et stabilité pour une démonstration publique.

7.4 Mise en production effective sur Neon, Render et Streamlit Cloud

L'analyse comparative de la section précédente a finalement été tranchée au profit d'une combinaison à trois étages, plus pragmatique pour une démonstration de fin de cursus que l'option Fly.io initialement envisagée. La base de données PostgreSQL est hébergée sur Neon dans la région de Francfort, le backend FastAPI est déployé sur Render via un blueprint Docker, et le tableau de bord Streamlit est servi par Streamlit Community Cloud directement depuis le dépôt GitHub. Cette répartition présente un coût d'hébergement strictement nul, chacun des trois fournisseurs offrant une formule gratuite suffisante pour l'usage de la maintenance.

Neon a été retenu plutôt qu'une base PostgreSQL embarquée chez Render parce que son offre gratuite ne met pas la base en veille, garantit trois gigaoctets de stockage et accepte un dump SQL chargé depuis un poste local avec la commande standard `psql`. La base est initialisée une seule fois en chargeant le dernier dump produit par l'ETL Talend : la commande `psql "$NEON_URL" -f talend/dump/mspr2_dump_*.sql` alimente Neon en quelques minutes avec l'intégralité des données ferroviaires du projet. Une vérification immédiate par un `SELECT COUNT(*) FROM trajet` confirme l'arrivée des enregistrements. Lors des prochaines mises à jour, l'ETL Talend continuera de tourner localement sur le poste de l'équipe, et seul le dump résultant sera rechargé dans Neon — Render ne supporte pas l'exécution d'un job Java planifié dans son palier gratuit.

Render lit automatiquement le fichier `render.yaml` ajouté à la racine du dépôt, qui définit un service web nommé `obrail-backend` de runtime Docker, en plan free, dans la région de Francfort, avec le dossier `backend` comme contexte de build et le chemin `/health` comme sonde de disponibilité. Deux variables d'environnement y sont déclarées en `sync false`, ce qui signifie qu'elles doivent être renseignées manuellement dans l'interface Render au premier déploiement : `DATABASE_URL` pointe vers Neon avec le suffixe `sslmode=require` obligatoire, et `CORS_ORIGINS` contient l'URL publique du dashboard Streamlit. Le mécanisme de blueprint évite toute saisie manuelle du Dockerfile, du healthcheck ou du port d'écoute : Render reconstruit l'environnement à l'identique sur n'importe quel autre compte simplement en relisant le fichier versionné.

Le tableau de bord est déployé sur `share.streamlit.io` en pointant vers la branche `main` du dépôt et le fichier `dashboard/app.py`. Les paramètres sensibles passent par le système de secrets de Streamlit Cloud, qui injecte un fichier TOML dans `st.secrets` au démarrage du conteneur. La clef `API_URL` contient l'URL publique du backend Render, et la clef `PROMETHEUS_URL` est laissée vide puisque la stack de supervision reste locale. Le client HTTP du dashboard a été modifié pour lire ces secrets en priorité avant de retomber sur la variable d'environnement, ce qui permet à la même base de code de fonctionner indifféremment en local avec `docker-compose` et en production sur Streamlit Cloud sans bifurcation logique. Un fichier `.streamlit/config.toml` versionné fixe en outre le thème visuel et la couleur primaire, et un fichier `secrets.toml.example` documente la liste des clefs attendues sans jamais committer leur valeur.

Cette mise en production a été l'occasion d'un durcissement du backend qui était indispensable pour exposer publiquement l'API sans risque opérationnel. Les sessions `SQLAlchemy` de la fonction `get_db` sont désormais encadrées par un bloc `try/finally` qui garantit la fermeture systématique de la connexion même si le handler de route lève une exception : sans cette précaution, une suite d'erreurs aurait progressivement épuisé le pool de connexions Neon, qui est dimensionné modestement dans le palier gratuit. La dépendance `slowapi`, qui n'était plus utilisée par aucune route depuis le retrait du `rate limiting` déclaratif, a été retirée du fichier `requirements.txt` afin de ne pas alourdir l'image Docker. La liste des origines CORS, jusqu'alors codée en dur, est maintenant lue depuis la variable d'environnement `CORS_ORIGINS` et acceptée sous la forme d'une liste séparée par des virgules, ce qui permet au même backend de servir simultanément le dashboard de production et un dashboard local en développement. Les schémas Pydantic des routes `Trajet` ont

reçu des contraintes `min_length` et `max_length` sur les champs `trajet_id`, `gare_depart` et `gare_arrivee`, afin que toute requête malformée soit rejetée par la couche de validation avant même d'atteindre le service métier.

Le fichier `docker-compose.yml` a parallèlement été modifié pour exiger explicitement les variables `POSTGRES_PASSWORD` et `GRAFANA_PASSWORD` au démarrage, sans valeur par défaut. Cette contrainte oblige tout nouvel environnement à fournir ses propres secrets, et empêche un déploiement accidentel avec les mots de passe d'exemple qui figuraient auparavant dans le fichier. Conjointement, le fichier `.gitignore` a été étendu pour exclure les fichiers de couverture de tests, le fichier `secrets.toml` de Streamlit et tout fichier `.codex` laissé par mégarde par un assistant de développement.

Le pipeline GitHub Actions a lui aussi été renforcé. La directive `continue-on-error` qui était positionnée sur les jobs de lint et de test, héritée d'une phase de mise en place où les pull requests pouvaient être fusionnées malgré une suite défailante, a été supprimée : un échec de lint ou de test bloque désormais réellement la livraison. Un délai d'expiration explicite a été ajouté sur le job de tests end-to-end Playwright, qui peut occasionnellement se figer si le navigateur Chromium ne démarre pas correctement dans le runner GitHub. Cette double mesure aligne le pipeline sur le contrat implicite annoncé au chapitre 5 : aucune branche ne fusionne sur `main` si l'ensemble des barrières de qualité n'est pas vert.

L'ensemble de la procédure de déploiement est documenté pas à pas dans le fichier `DEPLOY.md` ajouté à la racine du dépôt. Ce document détaille les sept étapes — création du projet Neon, chargement du dump, création du blueprint Render, renseignement des variables d'environnement, déploiement Streamlit, mise à jour réciproque des CORS, vérifications curl post-déploiement — et liste explicitement les services qui ne sont pas déployés et la raison technique pour chacun : Prometheus et Grafana restent locaux car leur exposition publique consommerait inutilement les paliers gratuits, et l'ETL Talend reste sur le poste de l'équipe car son moteur Java planifié n'a pas sa place sur Render free. L'endpoint `/metrics` du backend reste exposé sur Render pour démontrer l'instrumentation, même si aucun Prometheus distant ne le scrape. Le coût total mensuel de cette infrastructure de démonstration est de zéro euro, le compromis étant un cold start de l'ordre de trente secondes sur Render après quinze minutes d'inactivité, jugé acceptable pour une démonstration et explicitement mentionné en début de présentation.

Chapitre 8 Conclusion

8.1 Synthèse des compétences couvertes

Ce projet couvre l'ensemble des huit compétences définies par le référentiel RNCP36581 du titre Développeur en Intelligence Artificielle et Data Science.

La compétence d'**analyse du besoin** est couverte par le chapitre d'introduction, qui formalise le contexte d'ObRail Europe, identifie les contraintes techniques et réglementaires, et traduit les besoins métier en sept objectifs opérationnels mesurables.

La compétence de **conception de l'architecture** est couverte par le schéma des cinq services Docker et les justifications des choix techniques documentées dans chaque chapitre : PostgreSQL pour la base de données, FastAPI pour le backend, Streamlit pour le frontend, Prometheus et Grafana pour le monitoring.

La compétence de **coordination en contexte agile et MLOps** est couverte par le workflow Git structuré autour des branches, des Pull Requests et du pipeline CI/CD qui impose la validation des tests avant tout merge. Le monitoring continu et le fichier why.md documentant les décisions techniques s'inscrivent dans une démarche MLOps.

La compétence de **développement front-end, back-end et sécurité** est couverte par le backend avec ses treize endpoints, son rate limiting et ses quatre headers de sécurité, par le frontend avec ses trois pages et ses mesures RGAA, et par l'ETL Talend qui traite les sources de données hétérogènes.

La compétence d'**automatisation des tests** est couverte par les cent vingt-huit tests organisés en cinq types et exécutés automatiquement dans le pipeline GitHub Actions à chaque push.

La compétence de **livraison continue** est couverte par le build automatique des images Docker et leur push vers GitHub Container Registry sur la branche main, garantissant que l'image déployée est exactement celle qui a été testée.

La compétence de **supervision avec monitoring** est couverte par Prometheus et Grafana avec leur dashboard en six panneaux, par la page Supervision du tableau de bord qui expose les métriques en temps réel, et par la gestion de la reproductibilité de la configuration Grafana documentée dans why.md.

La compétence de **résolution des incidents** est couverte par les trois corrections documentées dans why.md : le problème de double montage du dump SQL au démarrage, le problème de l'UID aléatoire de Grafana qui rendait le dashboard vide sur d'autres machines, et les mesures de sécurité ajoutées suite à l'identification d'un backend non protégé.

8.2 Perspectives d'accueil d'un futur modèle d'intelligence artificielle

Pour situer précisément ce projet dans la progression vers un système d'intelligence artificielle en production, il est utile de le positionner sur le modèle de maturité MLOps de Google, qui définit trois niveaux de sophistication croissante.

Le niveau 0, dit MLOps manuel, correspond à une situation où les modèles sont entraînés et déployés à la main, sans automatisation ni supervision — c'est la situation du MSPR2. Le niveau 1, dit automatisation du pipeline ML, correspond à une situation où l'entraînement est déclenché automatiquement par de nouvelles données et où le modèle est supervisé en production. Le niveau 2, dit automatisation du pipeline CI/CD ML, correspond à une situation où la mise à jour du code du modèle déclenche automatiquement un entraînement, une validation et un déploiement, avec tests A/B et retour en arrière automatique.

Le MSPR3 place ObRail Europe en position de démarrer directement au niveau 1 lors de l'intégration du premier modèle. Le pipeline CI/CD GitHub Actions fournit déjà le squelette dans lequel une étape d'entraînement automatique peut être insérée. La base PostgreSQL stocke les données d'entraînement dans un format propre et normalisé prêt à être consommé par scikit-learn, PyTorch ou TensorFlow. Prometheus est déjà en place pour recevoir des métriques de dérive des données en ajoutant simplement de nouveaux compteurs dans le service d'inférence.

Le projet a été conçu dès l'origine pour faciliter l'intégration d'un modèle d'intelligence artificielle dans une prochaine itération. Plusieurs éléments de l'architecture actuelle constituent des fondations directement exploitables.

L'API expose les données ferroviaires au format JSON standardisé, ce que tout modèle d'apprentissage machine peut consommer facilement sans transformation supplémentaire. Le pipeline CI/CD peut être étendu pour intégrer une étape d'entraînement automatique déclenchée lors des modifications des données sources ou du code du modèle. Le monitoring Prometheus peut être étendu pour suivre des métriques spécifiques aux modèles d'intelligence artificielle, comme la dérive des données d'entrée, la précision des prédictions ou le taux de rappel, en ajoutant simplement de nouveaux compteurs dans le service d'inférence. Les volumes persistants Docker permettent de conserver les artefacts d'entraînement, les checkpoints de modèles et les historiques d'évaluation entre les redémarrages. La conteneurisation Docker garantit que le service d'inférence sera déployé dans les mêmes conditions reproductibles que le reste de la stack.

La prochaine étape naturelle du projet consiste à implémenter un microservice de prédiction qui, à partir des données historiques des trajets stockées dans PostgreSQL, estimera la fréquentation future d'une ligne ou recommandera des optimisations de dessertes. Ce service s'inscrirait dans le réseau Docker existant, exposerait ses propres endpoints via l'API FastAPI, et bénéficierait immédiatement du monitoring et de la couverture de tests mis en place dans ce MSPR3. En ce sens, la valeur de ce projet ne réside pas seulement dans ce qu'il fait aujourd'hui, mais dans la robustesse de l'infrastructure qu'il établit pour demain.

Lien du repo github : github.com/JXPM/MSPR